COVER-KBC v2

Coverage-guided Open-set Verification and Elicitation for Relation-wise Knowledge Base Construction

Implementation-Ready Architecture Specification

AKBC Shared Task 2026 @ EMNLP 2026

Architecture Thesis

Treat each subject–relation query as a **typed active evidence-acquisition problem**. A deterministic relation program controls how a frozen open-weight LLM discovers atomic candidates, how candidates are independently verified with calibrated logits, how evidence and uncertainty are accumulated, and how inference-time compute is adaptively allocated until the expected value of another action is too small.

Architecture Freeze Candidate v2.0

This document supersedes the earlier COVER-KBC concept note as the implementation specification. The previous Chao/unseen-cardinality estimator is **not a core dependency**; the core is now relation-typed programs, independence-aware evidence, logit-calibrated blind verification, residual-coverage signals, and adaptive control.

Root-repository design document – not the final 6-page system paper

August 2026

Contents

List of Figures

1 Executive Decision

COVER-KBC v2 is designed specifically for the AKBC Shared Task 2026. The official task provides a subject s and relation r and requires the system to output the complete set of correct object strings. Cardinality is not fixed: a query can have zero, one, or many objects. The challenge is closed-book, allows multi-step/agentic inference, prohibits external factual retrieval and additional model training, and imposes a 32B total inference-time neural-parameter budget [1, 2].

The key architectural decision is to **avoid a monolithic prompting strategy**. The six relations have fundamentally different error surfaces. COVER-KBC therefore compiles the relation into one of four typed inference programs and then runs an active loop over candidate discovery, verification, uncertainty reduction, and stopping.

Final high-level formulation

Relation program → diverse elicitation → atomic evidence graph
→ calibrated blind verification → adaptive controller → final set.

The central novelty is not "combining ReWiSe and Self-RAG." ReWiSe remains an important prior because it shows that relation-wise aggregation and repeated probing can outperform single generation [3]. The 2025 runner-up shows that self-generated relation-focused context and decomposition can expose latent parametric knowledge [4]. COVER-KBC changes the control problem: it explicitly tracks what kind of evidence supports each atomic object, how reliable that evidence is, what uncertainty remains, which acquisition action is worth executing next, and when further inference is likely to hurt rather than help.

Core components that are implementation requirements

- Exact relation compiler and four deterministic typed programs.
- Structurally diverse candidate discovery with provenance.
- Atomic candidate normalization and an independence-aware evidence graph.
- Blind three-way verifier using raw label logits plus contextual calibration.
- Residual Coverage & Saturation Estimator (RCSE), not a literal unseen-cardinality estimator.
- Rule-based active controller with relation-specific stopping.
- Evaluator-aware final selection and exact JSONL output.

High-priority experimental branches, not hard dependencies

- DoLa-style intermediate-layer factual decoding as an additional evidence view.
- Heterogeneous cross-model verification if a model bake-off shows a gain under the 32B budget.
- More sophisticated scheduler scores after the deterministic controller is stable.

2 Official Task Contract and Non-Negotiable Constraints

2.1 Task and evaluation

The challenge defines the target as

$$(s, r) \mapsto O^\star(s, r) = \{o_1, \ldots, o_k\}, \qquad k \in \{0, 1, 2, \ldots\}.$$

String relations are normalized and alias-matched; numeric relations are correct within a 5% relative tolerance. Evaluation reports macro precision, macro recall, and macro F1 over object sets [1, 2]. The six relations are:

Table 1: Official relations and their dominant structural type.

Relation	Program type	Critical semantic constraint
countryLandBordersCountry	SMALL_SET	physical land borders only; integral overseas territories may count
personHasCityOfDeath	NULL_SINGLE	city/locality; living or unknown locality means empty
hasCapacity	NUMERIC	highest published maximum spectator capacity
awardWonBy	LARGE SET	recipient entity, exact award, no rescinded awards
companyTradesAtStockExchange	SMALL_SET	company itself must be publicly traded on the exchange
hasArea	NUMERIC	total area in km^2, not land-only area

The July 2026 ground-truth update is especially important: the gold reflects the world as of 1 July 2026, including recent deaths, current stock listings, recent award winners, border corrections, richer aliases, and evaluator normalization changes [2]. Most award rows were cleaned and completed, but the official repository still notes that a small number of very large or open-ended awards necessarily have partial gold sets [2].

2.2 Rules that directly shape architecture

Table 2: Rule-to-design mapping.

Rule	Status	Architecture consequence
No web, RAG, external factual corpus, or KB lookup	Mandatory	All factual evidence must come from frozen model parameters; no hidden retrieval fallback.
No fine-tuning, continued pretraining, instruction tuning	Mandatory	No task-trained neural router/verifier. Use prompts, logits, deterministic calibration, and non-neural thresholds only.
Total inference-time neural budget $\leq$ 32B	Mandatory	Every model/component is counted; MoE uses total published parameters; quantization does not reduce counted size.
Multi-step/agentic inference allowed	Opportunity	Active controller and multi-view probing are legal.
Rule-based filtering/normalization/aggregation allowed	Opportunity	Relation compiler, evidence graph, numeric clustering, deduplication, and scheduler can be deterministic.
Public code required; system paper max 6 main pages	Deliverable constraint	Architecture must be reproducible, easy to ablate, and explainable with a compact narrative.

Parameter-budget audit rule

Do not infer challenge compliance from a model's marketing suffix alone. Before a model profile is enabled, record the official published parameter count for every neural component actually used. If a multimodal checkpoint has a separate vision encoder or the model card and artifact metadata

disagree, use the conservative count or ask the organizers for clarification. No model pairing is considered "frozen" until this audit passes.

2.3 Official baseline as a diagnostic

The official Qwen3.5-9B validation baseline currently reports overall macro-F1 0.313 [2]. More important than the aggregate are the relation-level patterns:

Table 3: Official validation baseline and the architectural diagnosis.

Relation	P	R	F1	Primary diagnosis
`awardWonBy`	.247	.078	.101	severe coverage/recall failure
`companyTradesAtStockExchange`	.369	.725	.354	recall is much better than precision; verification matters
`countryLandBordersCountry`	.697	.911	.665	already strong recall; preserve precision and stop early
`hasArea`	.290	.290	.290	factual numeric recall instability
`hasCapacity`	.180	.180	.180	factual numeric recall + semantic ambiguity
`personHasCityOfDeath`	.210	.600	.210	null/existence handling is weak; baseline emits no empty predictions

This table is why a single fixed inference strategy is rejected.

3 Research Objective and Control Problem

Let π denote the non-neural inference policy that decides what action to execute next. COVER-KBC optimizes the conceptual objective

$$\pi^\star = \arg\max_{\pi} \mathbb{E}\left[F_1\left(\widehat{O}_\pi, O^\star\right) - \lambda C_\pi - \rho R_\pi\right],$$

where C_π is inference cost and R_π is redundant compute. The redundancy term explicitly penalizes repeated calls that probe essentially the same mode without adding independent evidence.

At time t, the controller state is

$$x_t = (G_t, \mathcal{U}_t, q_t^{\text{res}}, Y_t, B_t, \mathcal{H}_t),$$

where:

- G_t is the candidate–evidence graph;
- $\mathcal{U}_t$ is the unresolved-candidate set;
- $q_t^{\text{res}} \in [0,1]$ is the residual-coverage need-to-continue signal;
- Y_t stores marginal yield statistics for views/facets;
- B_t is the remaining token/call budget;
- $\mathcal{H}_t$ stores uncertainty and disagreement statistics.

The next action is

$$a_t = \pi(x_t),$$

with actions such as `run_direct`, `run_facet`, `blind_verify`, `run_DoLa`, `cross_model_check`, or `stop`. The competition implementation starts with a deterministic policy, not reinforcement learning.

4 COVER-KBC v2: Overall Architecture

4.1 Four-plane view

The system is easiest to reason about as four planes.

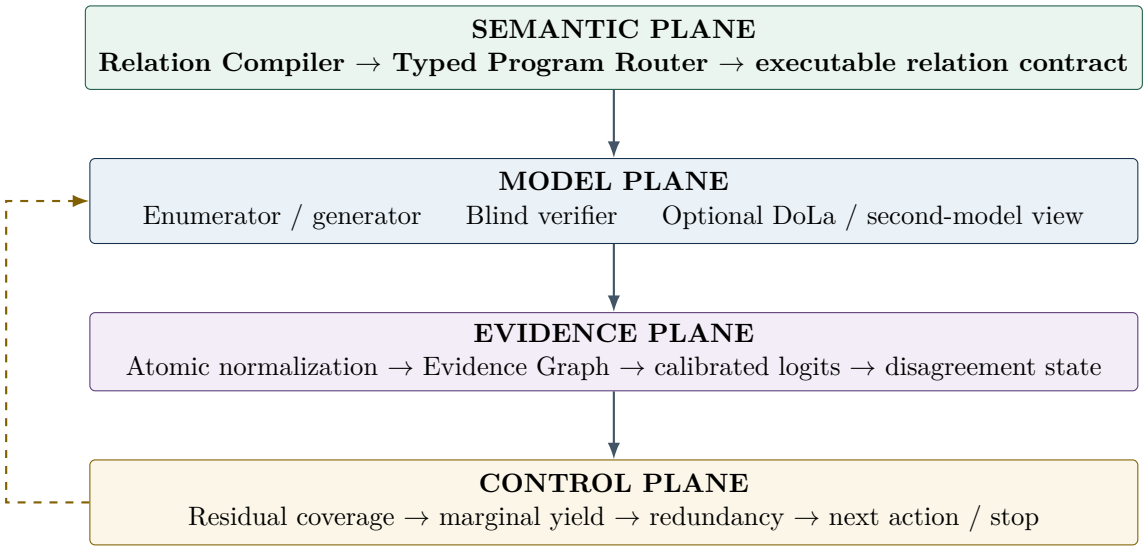

Figure 1: COVER-KBC v2 organized into semantic, model, evidence, and control planes.

4.2 End-to-end runtime flow

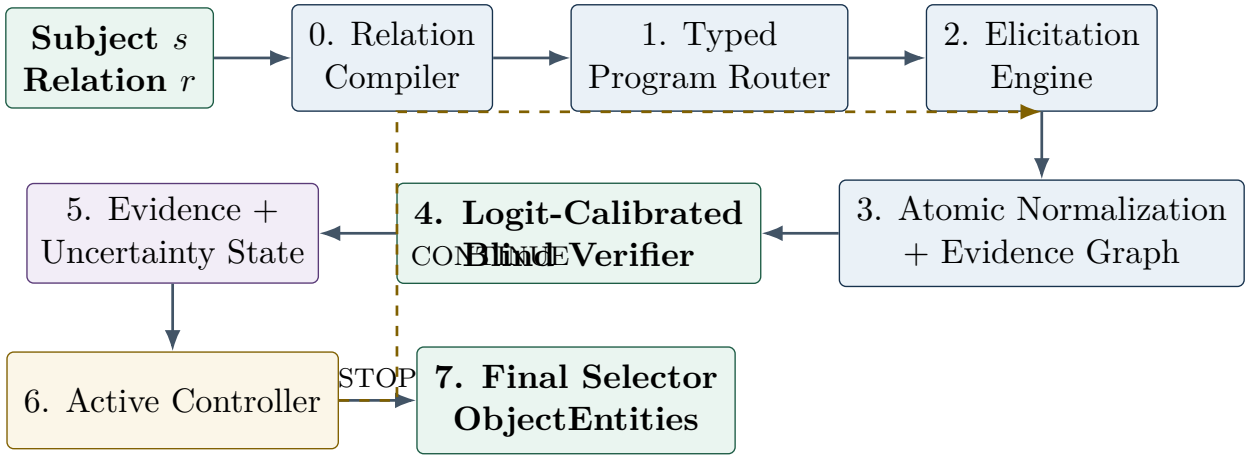

Figure 2: Inference is a stateful loop, not a one-pass pipeline.

A query always begins with the relation compiler and mandatory initial views. After that, the verifier and controller operate only where needed. Easy border queries should terminate quickly; difficult awards are allowed to consume a much larger share of the budget.

5 Module 0 – Relation Compiler

5.1 Purpose

The Relation Compiler converts an official relation ID into an executable semantic contract. It is non-neural and must be implemented before any sophisticated prompting. The contract specifies:

- answer type and cardinality regime;

- exact positive semantics;
- hard negative / near-miss classes;
- mandatory and optional views;
- parser and normalizer;
- verification templates;
- relation-specific stopping and final-selection policy.

The design is motivated by the observation that systematic knowledge coverage and reasoning are distinct capabilities: KnowledgeBerg shows large gaps in universe enumeration and knowledge-grounded reasoning across open models [5]. COVER-KBC therefore avoids asking the model to rediscover relation semantics on every call.

5.2 Example contract

```
relation: companyTradesAtStockExchange
program_type: SMALL_SET
answer_type: exchange_entity
cardinality: zero_or_many
positive_rule:
  - the subject company itself is publicly traded on the exchange
hard_negatives:
  - parent company is listed but subject is not
  - subsidiary is listed but subject is not
  - exchange is merely mentioned in company history
mandatory_views:
  - listing_gate
  - direct_exchange_recall
optional_views:
  - primary_secondary_listing
  - blind_candidate_check
final_policy: high_precision_small_set
```

5.3 Why this is a compiler, not a prompt file

The contract is consumed by several modules simultaneously. It controls the prompt template, the parser, the evidence types, the candidate verifier, the action space, and the stopping rule. The same contract also supplies test fixtures. A change in an official relation definition should be made once in the contract and then propagate deterministically.

6 Module 1 – Typed Program Router

The router is a deterministic mapping from relation to an inference program. No learned classifier is needed because only six relation IDs exist.

Table 4: Four typed programs.

Program	Relations	Core behavior
SMALL SET	borders, stock exchange	small candidate universe; precision-aware verification; fast stopping
NULL/SINGLE	city of death	existence gate first; zero-or-one final object; explicit abstention
NUMERIC	area, capacity	semantically diverse scalar recalls; deterministic normalization; robust clustering
LARGE OPEN SET	awards	recall-first facet exploration followed by atomic verification and saturation control

The design takes inspiration from SELF-DISCOVER, which shows that task-specific reasoning structures can outperform one universal chain-of-thought pattern [8]. COVER-KBC makes the structure deterministic because the relation taxonomy is fixed and small; this is cheaper, reproducible, and easier to ablate.

7 Module 2 – Diverse Elicitation Engine

7.1 Purpose

The Elicitation Engine is a **candidate generator**, not the final answer generator. It is deliberately allowed to be noisy. The final system separates high-recall discovery from high-precision verification, a pattern also supported by work on multi-answer QA and atomic answer merging [6, 7].

7.2 View families

Every model call has a `view_id`, `independence_group`, prompt template, decoding profile, and cost record.

Table 5: View families used by the elicitation engine.

View	Purpose	Example
Direct	cheapest baseline factual recall	"List all countries sharing a land border with X."
Structural	force a task-specific decomposition	compass directions for borders; primary/secondary listings for stock
Relation-focused description	trigger parametric memory before extraction	describe only the geographic boundary / trading status / death locality
Contrastive	separate exact targets from near misses	land vs maritime; total area vs land area; recipient vs winning work
Facet	search one coherent subspace	award decade, recipient type, category, discipline
Missingness	search only for additional objects	"Do not repeat the verified set; identify a missing facet or object."
Reverse / alternate framing	obtain structurally different evidence	"Does candidate Y share a land border with X?"
Factual-decoding	optional DoLa-style decoding	decode using later-vs-earlier layer contrast
Cross-model	optional heterogeneous evidence	second model sees no generator rationale

7.3 Structural diversity beats pure repetition

ReWiSe demonstrated that repeated stochastic generations plus entity-level aggregation can greatly improve over a single generation [3]. However, self-consistent errors can remain stable even across stochastic samples, and different model families can make different persistent mistakes [13]. COVER-KBC therefore distinguishes:

$$\text{raw frequency} \quad \neq \quad \text{independent evidence diversity.}$$

Ten repeats of the same direct view belong to one evidence family; direct + structural + contrastive + reverse views represent four qualitatively different supports.

7.4 Provenance record

```
{
  "subject": "Germany",
  "relation": "countryLandBordersCountry",
  "view_id": "border_compass_v1",
  "independence_group": "STRUCTURAL_GEOGRAPHY",
  "run_id": 2,
  "raw_output": "...",
  "parsed_candidates": ["Denmark", "Poland", "Czechia"],
  "generated_tokens": 126,
  "latency_ms": 812,
  "decode_profile": "sample_t0.4"
}
```

No downstream module should consume a candidate without provenance.

8 Experimental Branch – DoLa-Style Factual Decoding

DoLa contrasts vocabulary projections from later and earlier Transformer layers to increase factuality without retrieval or fine-tuning [9]. Conceptually, for hidden state $h_t^{(\ell)}$ at layer ℓ and LM head W, define

$$z_t^{(\ell)} = W h_t^{(\ell)}.$$

A simplified contrastive score is

$$z_t^{\text{contrast}} = z_t^{(L)} - z_t^{(\ell)},$$

where L is the mature layer and ℓ is a selected premature layer. The original method contains additional layer-selection and plausibility details; implementation must follow the paper rather than treating this simplified equation as a complete reimplementation.

In COVER-KBC, DoLa is **not the default decoder**. It is an optional evidence-producing view:

$$\text{standard recall} + \text{DoLa recall} + \text{structural recall.}$$

The branch is retained only if it improves AKBC validation under matched or clearly reported compute. It must be disabled cleanly by configuration.

Compatibility requirement

DoLa requires access to intermediate hidden states and the model's LM head. Some optimized serving engines may not expose these cheaply. Therefore the system architecture must not depend on DoLa for correctness or basic operation.

9 Module 3 – Atomic Normalization and Candidate–Facet Evidence Graph

9.1 Atomic candidates

All textual generations are parsed into relation-typed atomic candidates. For string relations, surface normalization is for internal deduplication only; the final output should retain one preferred human-readable surface form. For numeric relations, candidates are parsed into numeric values plus units.

9.2 Graph representation

Let o denote a normalized candidate and e an evidence event. The graph stores signed edges

$$e \to o \in \{\text{SUPPORT}, \text{CONTRADICT}, \text{UNKNOWN}\}.$$

Each edge contains `independence_group`, model ID, view ID, run ID, raw verifier distribution if available, and cost.

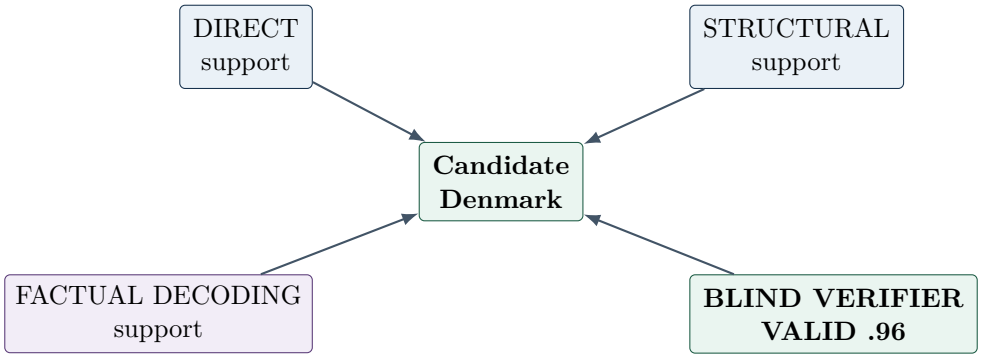

Figure 3: The graph records independent evidence types rather than treating every generation as an equal vote.

9.3 Hard contract rules

Some candidates can be rejected without another neural call. Examples include a country/region string for `personHasCityOfDeath` when the output must be a city/locality, a non-numeric output for `hasArea`, or an obvious duplicate surface form. Hard semantic rejection rules must be conservative; they may remove type/format impossibilities but must not encode external factual lookup.

10 Module 4 – Logit-Calibrated Blind Verifier

This module is the primary use of autoregressive logits in COVER-KBC.

10.1 Why not use the probability of the whole candidate string?

Sequence likelihood is strongly affected by tokenization, name length, orthography, and lexical frequency. The probability of "New York Stock Exchange" is therefore not a clean estimate of factual correctness. Instead, verification is converted to a small classification problem with fixed labels.

10.2 Three-way verification

For candidate o, the verifier sees only the subject, exact relation contract, candidate, and three labels:

$$A = \text{VALID}, \qquad B = \text{INVALID}, \qquad C = \text{UNKNOWN}.$$

The generator's explanation is deliberately hidden. This follows the independence principle in Chain-of-Verification: verification questions should be answered independently to reduce anchoring on the draft [7].

Let raw first-token label logits be z_A, z_B, z_C. The uncalibrated distribution is

$$p_j = \frac{\exp(z_j)}{\exp(z_A) + \exp(z_B) + \exp(z_C)}.$$

10.3 Contextual calibration

Few-shot and prompted classifiers can have strong label priors. Inspired by contextual calibration [10], run the same verifier template with a content-free control instance and obtain bias logits b_A, b_B, b_C. Define

$$\widetilde{z}_j = z_j - b_j, \qquad \widetilde{p}_j = \mathrm{softmax}\left(\frac{\widetilde{z}_j}{T}\right).$$

The default implementation uses $T = 1$; fitting a learned calibrator is deliberately avoided. If any task-specific temperature fitting is considered later, it must be treated as a rule-compliance question first.

Useful signals are:

$$P_V(o) = \widetilde{p}_A,$$

$$M(o) = \widetilde{z}_A - \max(\widetilde{z}_B, \widetilde{z}_C),$$

and

$$H_{\mathrm{ver}}(o) = -\sum_{j \in \{A,B,C\}} \widetilde{p}_j \log \widetilde{p}_j.$$

10.4 Prompt-distribution stability

MONITOR measures factual reliability through distribution changes under prompt/context variation [11]. COVER-KBC adopts the same principle at candidate level. If $p_1(o), \ldots, p_m(o)$ are verifier distributions from independent templates, let

$$\bar{p}(o) = \frac{1}{m}\sum_{i=1}^{m} p_i(o),$$

and define a disagreement score

$$U_{\mathrm{prompt}}(o) = \frac{1}{m}\sum_{i=1}^{m} D_{\mathrm{KL}}\left(p_i(o) \,\|\, \bar{p}(o)\right).$$

High disagreement makes a candidate unresolved even when one prompt greedily emits VALID.

10.5 Verification policy

Do not verify every candidate equally.

- **Auto-accept tier**: strong independent support, no contradiction, low uncertainty.
- **Verifier tier**: weak support, score near threshold, or disagreement across views.
- **Adversarial verifier tier**: candidate is a common near miss specified by the contract.
- **Auto-reject tier**: hard type/format violation.

11 Module 5 – Evidence and Uncertainty State

The controller should not make decisions from a single scalar confidence. The state combines multiple signals.

11.1 Independent-support coverage

Let $g(o)$ be the number of eligible independent evidence groups that support candidate o, and $m(o)$ the number of groups capable of expressing that candidate. Define

$$q(o) = \frac{g(o)}{m(o)}.$$

A simple inclusion-uncertainty term is

$$H_{\text{inc}}(o) = -q(o)\log q(o) - (1-q(o))\log(1-q(o)).$$

This borrows the motivation of semantic-uncertainty work: uncertainty should respect semantic equivalence rather than raw token variation [12].

11.2 Candidate score

A deterministic v1 score is

$$S(o) = \alpha F(o) + \beta L(o) + \gamma X(o) - \delta C(o) - \eta U(o),$$

where:

- $F(o)$ = independent facet/view support;
- $L(o)$ = calibrated log-odds from the verifier;
- $X(o)$ = optional cross-model support;
- $C(o)$ = explicit contradiction count/strength;
- $U(o)$ = prompt/view disagreement.

One possible logit term is

$$L(o) = \text{clip}\left[\log \frac{P_V(o)+\epsilon}{P_I(o)+P_U(o)+\epsilon}\right].$$

Hard contract violations bypass the score and reject the candidate.

11.3 No learned neural fusion

The first implementation must not train a classifier on graph features. Weights and thresholds are human-designed and adjusted as ordinary non-neural system hyperparameters using train/internal validation. Keep the number of free parameters small enough to avoid overfitting ten-example relations such as `awardWonBy`.

12 Module 6 – Residual Coverage & Saturation Estimator (RCSE)

12.1 Why v1 Chao estimation is no longer core

The earlier design attempted to estimate the unseen object count with capture–recapture statistics. This is risky because model-generated views are not independent captures, and the official repository states that a small number of very large/open-ended award rows have necessarily partial gold sets

[2]. A literal estimate of the "true real-world set size" is therefore not aligned with the leaderboard objective.

COVER-KBC v2 replaces it with a weaker and more useful quantity:

$$q_t^{\text{res}} \in [0, 1],$$

interpreted as **need to continue searching**, not "probability that exactly n objects remain."

12.2 Signals

RCSE consumes relation-specific signals such as:

- number of newly verified candidates in recent actions;
- marginal verified yield per generated token;
- overlap between recent candidate sets;
- unresolved-candidate mass;
- mandatory-facet completion;
- verifier disagreement;
- set stability;
- relation-specific gates (e.g. death status resolved, numeric cluster stable).

For large-set search, define marginal yield of action a_t as

$$Y_t(a_t) = \frac{\#\text{new verified candidates from } a_t}{\text{generated tokens}(a_t) + \epsilon}.$$

A simple saturation statistic over the last k actions is

$$\text{Sat}_t = 1 - \frac{1}{k} \sum_{i=t-k+1}^{t} \mathbf{1}[\Delta V_i > 0],$$

where ΔV_i is the number of newly verified candidates. High saturation means recent calls are not increasing the verified set.

12.3 Relation-typed residuals

Table 6: RCSE is intentionally relation-specific.

Program	Residual signal
SMALL_SET	mandatory views complete + high set overlap + no disputed candidate + low new-candidate yield
NULL_SINGLE	existence gate resolved + zero or one stable locality + no competing high-score locality
NUMERIC	dominant cluster stable + low relative dispersion + independent semantic/unit views agree
LARGE_OPEN_SET	verified-yield decay + facet coverage + missingness queries stop adding trusted objects + low unresolved tail

13 Module 7 – Active Controller and Adaptive Stopping

Adaptive test-time compute is motivated by work showing that optimal inference-time strategies vary with problem difficulty and that per-query compute allocation can outperform fixed best-of-N strategies [14]. COVER-KBC uses a deterministic controller for reproducibility and rule safety.

13.1 Action score

For each legal action a, estimate

$$A_t(a) = \alpha \widehat{Y}_t(a) + \beta G_t(a) + \gamma U_t(a) - \lambda C(a) - \rho D_t(a),$$

where:

- $\widehat{Y}_t(a)$ = expected marginal verified-candidate yield;
- $G_t(a)$ = relevance to an uncovered relation facet or unresolved candidate;
- $U_t(a)$ = uncertainty reduction expected from the action;
- $C(a)$ = expected token/call/latency cost;
- $D_t(a)$ = redundancy with evidence already acquired.

The controller selects

$$a_t^\star = \arg\max_{a \in \mathcal{A}_t} A_t(a)$$

unless a relation-specific stopping rule fires first.

13.2 Rule-based policy v1

1. Run each mandatory initial view once.
2. Update graph and deterministic hard filters.
3. Verify high-impact uncertain candidates before spending budget on more discovery.
4. If a semantic/facet gap exists, run the highest-priority targeted view.
5. If large-set marginal yield remains high, continue exploration.
6. If yield collapses but uncertainty remains, prefer verification over more enumeration.
7. If all relation-specific stability criteria hold, stop.
8. Always stop at the hard call/token budget.

13.3 Set stability

For accepted set A_t, track

$$J_t = \frac{|A_t \cap A_{t-1}|}{|A_t \cup A_{t-1}|}.$$

High J_t is useful but never sufficient alone: a wrong or incomplete set can also be stable.

14 Module 8 – Final Selector and Evaluator-Aware Output

The final selector maps graph state to `ObjectEntities`. It must optimize the actual evaluator, not linguistic elegance.

14.1 String relations

A candidate is emitted if it exceeds the relation threshold and has no hard violation. Emit only one preferred surface form per semantic candidate. The official evaluator uses maximum bipartite alias matching and warns that multiple surface forms of the same entity can reduce precision [2].

14.2 Numeric relations

Numeric outputs are selected from the dominant robust cluster rather than by token likelihood. For normalized values x_i, define relative distance

$$d(x_i, x_j) = \frac{|x_i - x_j|}{\max(|x_i|, |x_j|)}.$$

Cluster using a conservative threshold related to, but not automatically identical to, the official 5% tolerance. The representative is typically

$$\hat{x} = \text{median}(C_{\text{dominant}}).$$

Track relative MAD as a stability signal:

$$\text{rMAD} = \frac{\text{MAD}(C_{\text{dominant}})}{|\,\text{median}(C_{\text{dominant}})| + \epsilon}.$$

15 Relation Program A – countryLandBordersCountry

This relation already has high baseline recall, so the architecture is intentionally small.

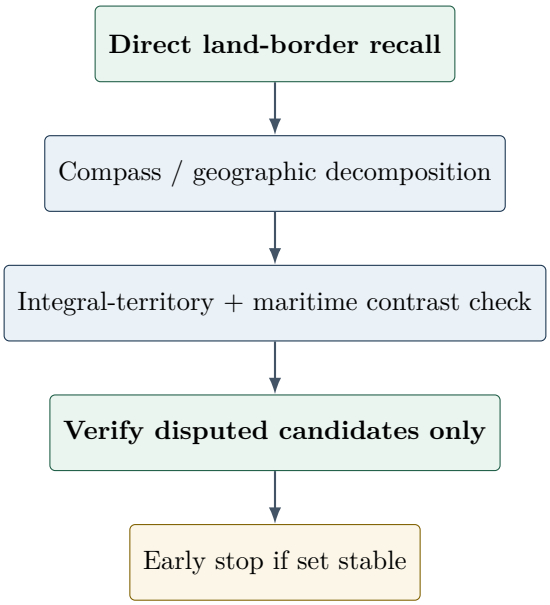

Figure 4: Border program.

Important official edge cases include borders through integral overseas territories and enclave borders, while maritime borders and non-integral dependencies are excluded [2]. The program should not waste repeated samples on easy mainland countries after the direct and structural views agree.

16 Relation Program B – personHasCityOfDeath

This program begins with an existence/null gate, because the official baseline outputs no empty predictions despite the relation explicitly allowing empty answers [2].

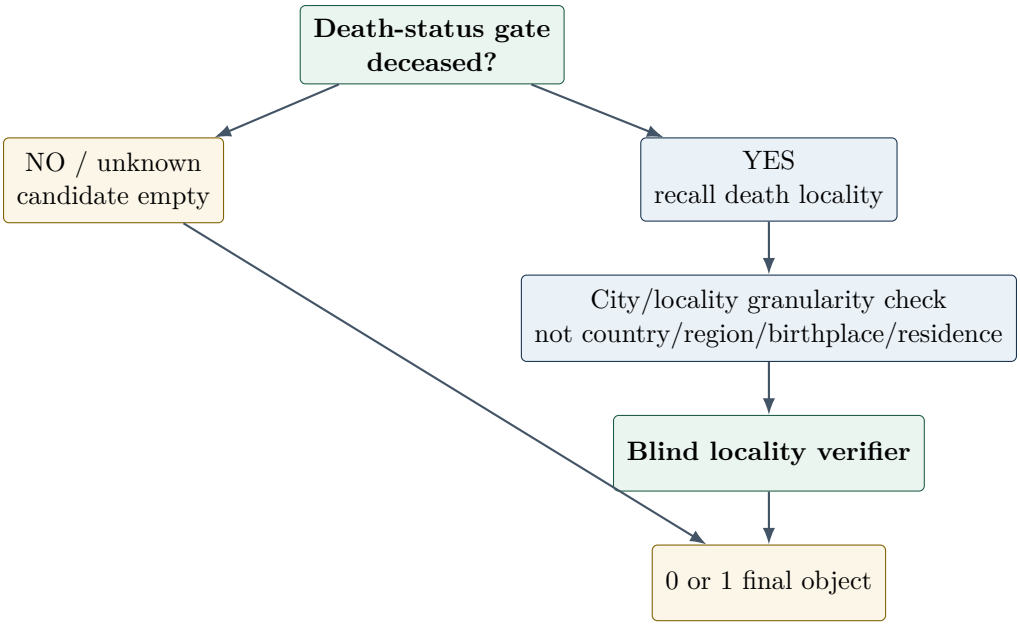

Figure 5: Death-city program.

For living/unknown cases, empty output is a first-class prediction, not a parser failure. The verifier should have an explicit UNKNOWN label rather than forcing a city guess.

17 Relation Program C – companyTradesAtStockExchange

The baseline has relatively high recall and low precision, so verification is more important than enumeration.

1. **Public-listing gate**: is the subject company itself publicly traded?
2. **Direct exchange recall**: generate likely exchanges only if the gate is positive/uncertain.
3. **Primary/secondary listing view**: search multiple legitimate listings without conflating parent/subsidiary companies.
4. **Atomic exchange verification**: verify each candidate independently with the exact company-vs-parent contract.
5. **High-precision final selection**: weak unresolved candidates are normally dropped.

18 Relation Program D – hasArea

Area uses semantic and unit diversity rather than repeated digit sampling.

1. Direct total-area recall in km^2.
2. Contrast “total area” against “land area” explicitly.
3. Independent alternate-unit recall (e.g. square miles or hectares) where model knowledge plausibly supports it.
4. Deterministically convert every value to km^2.
5. Cluster by relative distance and select a robust median.
6. Stop when the dominant cluster is stable and cross-unit views agree.

Cross-unit agreement gives a meaningful consistency check without trusting the probability of a digit string.

19 Relation Program E – hasCapacity

Capacity is similar numerically but semantically harder because multiple capacities can be plausible. The official target is the highest published maximum spectator capacity [1, 2].

Use views that specifically contrast:

- maximum spectator capacity;
- seated vs total capacity;
- current vs pre/post-renovation capacity;
- record attendance vs actual venue capacity.

After normalization to integers, cluster values and prefer a cluster supported by multiple semantic formulations. Record-attendance values should be treated as a contract near miss rather than a second valid target.

20 Relation Program F – awardWonBy

Awards receive the largest adaptive budget because the official baseline shows severe recall failure and some subjects have hundreds of recipients [2].

20.1 Dynamic facet search

The program begins with direct enumeration and then activates facets only when marginal yield justifies them:

- temporal ranges;
- recipient type: people, groups, organizations, projects;
- discipline/category where the award semantics support it;
- geography/nationality only when useful, never as a universal fixed decomposition;
- missingness prompts conditioned on already verified recipients.

Unlike the fixed multi-slice approach of the 2025 runner-up [4], the controller stops exploring a facet when it repeatedly produces no new verified objects.

20.2 Precision protection

Every weak/new candidate is checked against near-miss rules:

- exact award identity, not similarly named predecessor/successor;
- recipient entity, not the winning book/album/work;
- no rescinded recipient;
- recipient type may be person, group, organization, or project.

Because a small number of open-ended awards have partial gold, low-confidence long-tail expansion is especially dangerous. RCSE should stop before the tail becomes dominated by one-off uncertain names.

21 Autoregressive Runtime and Logit Access

21.1 Where logits come from

For a decoder-style causal LM, tokenized prompt $x_{1:n}$ is mapped through embeddings and successive hidden transformations:

$$h^{(0)} \to h^{(1)} \to \cdots \to h^{(L)}.$$

The final LM head produces

$$z_t = W_{\mathrm{LM}} h_t^{(L)} + b,$$

and token probabilities are

$$p(x_{t+1} = v \mid x_{\le t}) = \mathrm{softmax}(z_t)_v.$$

COVER-KBC needs raw logits for the verifier labels and, if DoLa is enabled, selected intermediate hidden states before final decoding.

21.2 Engineering requirement

The model runtime interface must support two modes:

1. **Generate mode**: text + optional token log-probabilities + token/cost accounting.
2. **Score mode**: no free-form generation; return logits for specified next-token label candidates A/B/C under a fixed verifier prompt.

This separation is essential for efficient verification and reproducibility.

22 Model Strategy: Bake Off Before Freeze

Architecture and model choice must remain decoupled. The system should expose a common runtime interface so the same COVER logic can be evaluated with multiple open-weight checkpoints.

22.1 Why model role specialization is plausible

KnowledgeBerg reports large differences between universe enumeration and reasoning across open models [5]. This suggests that the model best at high-recall enumeration may differ from the model best at compact verification. Self-consistent-error work further motivates heterogeneous evidence because persistent errors can differ across model families [13].

22.2 Candidate profiles

Table 7: Model profiles to benchmark; none is frozen by this document.

Profile	Candidate role	Why test it	Budget status
Single large Qwen3.5-class checkpoint	generator + verifier	strong reasoning/instruction following; simplest logit infrastructure	verify exact published total before enabling
Mistral Small 3.2 24B-class checkpoint	enumerator	strong candidate for broad factual enumeration; Apache-2.0 model card	verify total including any non-text neural components
Mistral enumerator + small Qwen verifier	heterogeneous roles	cross-family diversity and specialized verification	**provisional**; sum conservative published totals and stay $\le$ 32B

Profile	Candidate role	Why test it	Budget status
Gemma 3 27B-class checkpoint	balanced control	alternative model family for bake-off	verify exact published total and license/use conditions

Mistral Small 3.2 is published as a 24B model and Apache-2.0 [15]. Qwen3.5-27B reports a 27B language model and Apache-2.0 [16]. The Qwen3.5-4B card reports a 4B language model while the hosting artifact metadata may expose a larger aggregate file/model count; this is exactly why the challenge-specific parameter audit must be conservative [17].

No model assumption in the architecture

The code must not hard-code prompts, token IDs, hidden-layer counts, or label tokens for one family. Model-specific adapters belong behind a runtime interface. A candidate model is accepted only after: (1) budget audit, (2) baseline run on official validation, (3) relation-level analysis, and (4) logit/verifier compatibility test.

22.3 Minimal bake-off protocol

For each candidate profile, run the same minimal relation-aware baseline and report:

- official macro P/R/F1 overall and per relation;
- null accuracy / empty prediction behavior;
- numeric tolerance accuracy;
- average candidate count;
- generated tokens and latency per query;
- time-sensitive subset performance for death, stock, and awards;
- verifier-label calibration stability.

Only after this bake-off should the production model profile be frozen.

23 Software Interfaces

23.1 Core data structures

```
@dataclass
class QuerySpec:
    subject: str
    relation: str
    program_type: str
    answer_type: str
    positive_rules: list[str]
    hard_negative_rules: list[str]
    mandatory_views: list[str]
    optional_views: list[str]

@dataclass
class EvidenceEvent:
    candidate_id: str
    edge_type: str          # SUPPORT / CONTRADICT / UNKNOWN
    view_id: str
    independence_group: str
    model_id: str
    run_id: str
    valid_prob: float | None
    invalid_prob: float | None
```

```
    unknown_prob: float | None
    token_cost: int

@dataclass
class CandidateState:
    canonical_value: str
    display_value: str
    supports: list[EvidenceEvent]
    contradictions: list[EvidenceEvent]
    score: float
    status: str             # ACCEPTED / REJECTED / UNRESOLVED

@dataclass
class ControllerState:
    accepted: list[str]
    unresolved: list[str]
    residual_coverage: float
    recent_yield: dict[str, float]
    budget_calls_left: int
    budget_tokens_left: int
```

23.2 Runtime API

```
class FrozenLMRuntime(Protocol):
    def generate(self, request: GenerationRequest) -> GenerationResult: ...
    def score_labels(self, request: LabelScoreRequest) -> LabelScoreResult: ...

class RelationProgram(Protocol):
    def initial_actions(self, query: QuerySpec) -> list[Action]: ...
    def legal_actions(self, state: ControllerState) -> list[Action]: ...
    def should_stop(self, state: ControllerState) -> bool: ...
    def finalize(self, state: ControllerState) -> list[str]: ...
```

The controller, contracts, graph, parsers, clustering, and output writer remain model-agnostic.

24 Repository Layout

The root repository should keep the official benchmark snapshot isolated from the implementation. Recommended layout:

```
.
|-- COVER_KBC_V2_ARCHITECTURE_SPEC.pdf
|-- COVER_KBC_V2_ARCHITECTURE_SPEC.tex
|-- README.md
|-- benchmark/                  # official snapshot; do not modify in-place
|-- configs/
|   |-- models/
|   |-- experiments/
|   `-- relations/
|-- prompts/
|   |-- discovery/
|   `-- verifier/
|-- src/
|   `-- cover_kbc/
|       |-- contracts/
|       |-- programs/
|       |-- runtime/
```

```
|       |-- discovery/
|       |-- normalization/
|       |-- evidence/
|       |-- verification/
|       |-- coverage/
|       |-- controller/
|       |-- selection/
|       `-- evaluation/
|-- scripts/
|   |-- run_baseline.py
|   |-- run_cover.py
|   |-- evaluate_local.py
|   `-- audit_model_budget.py
|-- tests/
|   |-- unit/
|   `-- integration/
`-- outputs/                    # ignored generated artifacts
```

Do not place raw model weights, large caches, generated outputs, or secret/local paths under version control.

25 Implementation Build Order

The architecture is ambitious, but implementation should proceed through a few coherent milestones rather than dozens of tiny phases.

25.1 Milestone 1 – Reproducible foundation and model bake-off

Build first:

- official data/evaluator wrapper;
- exact baseline reproduction;
- model runtime abstraction;
- deterministic run manifest (seed, model revision, prompt hash, config hash);
- token/latency/call accounting;
- conservative model-parameter budget audit;
- minimal model bake-off.

Exit criterion: official validation can be reproduced from one command, every prediction is traceable to a run manifest, and at least one compliant model profile is selected for development.

25.2 Milestone 2 – COVER-Core

Implement:

- six Relation Contracts;
- four typed programs;
- mandatory multi-view discovery;
- parsers/normalizers;
- Candidate–Facet Evidence Graph;
- A/B/C logit verifier and contextual calibration;
- relation-specific final selector.

Run with a **fixed budget** first. This isolates whether contracts + diverse evidence + logit verification actually improve F1 before adding adaptive control.

Exit criterion: COVER-Core beats or clearly improves important relation-level failure modes relative to the direct baseline under a transparent fixed budget.

25.3 Milestone 3 – Active control and high-value experimental branches

Implement:

- RCSE;
- rule-based scheduler;
- adaptive stopping;
- award dynamic facet search;
- numeric stability logic;
- optional DoLa and cross-model branches behind feature flags.

Exit criterion: adaptive system improves F1/compute or matched-budget F1 over fixed-budget COVER-Core. Experimental branches are kept only when validation evidence justifies them.

26 Inference Algorithm

Algorithm 1 COVER-KBC v2 inference for one subject–relation query

Require: subject s, relation r, budget B
$q \leftarrow \textsc{CompileRelation}(s, r)$
$P \leftarrow \textsc{RouteProgram}(q)$
$G \leftarrow$ empty evidence graph
$S \leftarrow \textsc{InitState}(q, P, B)$
for all $a \in P.\textsc{InitialActions}(q)$ **do**
    $y \leftarrow \textsc{Execute}(a)$
    $C \leftarrow \textsc{ParseNormalize}(y, q)$
    $G \leftarrow \textsc{UpdateGraph}(G, a, C, y)$
    $S \leftarrow \textsc{UpdateState}(S, G, y)$
end for
while $\neg P.\textsc{ShouldStop}(S)$ and $\neg S.\textsc{BudgetExhausted}()$ **do**
    $u \leftarrow \textsc{HighestImpactUnresolved}(G, q)$
    if $u \neq \varnothing$ and $\textsc{WorthVerifying}(u, S)$ **then**
        $v \leftarrow \textsc{BlindVerifyWithLogits}(u, q)$
        $G \leftarrow \textsc{UpdateGraph}(G, v)$
    else
        $S.q_{res} \leftarrow \text{RCSE}(S, G, P)$
        $a \leftarrow \textsc{ChooseAction}(S, G, P)$
        if $a = \textsc{Stop}$ **then**
            break
        end if
        $y \leftarrow \textsc{Execute}(a)$
        $C \leftarrow \textsc{ParseNormalize}(y, q)$
        $G \leftarrow \textsc{UpdateGraph}(G, a, C, y)$
    end if
    $S \leftarrow \textsc{UpdateState}(S, G)$
end while
return $P.\textsc{Finalize}(S, G)$

27 Experiment and Ablation Plan

27.1 Baselines

At minimum, compare against:

1. official Qwen3.5-9B baseline;
2. direct relation-aware single generation;
3. ReWiSe-style fixed multi-sampling + entity frequency aggregation;
4. description-first/self-generated-context baseline;
5. COVER-Core fixed budget;
6. full COVER-KBC adaptive budget.

27.2 Ablation ladder

Table 8: Primary ablations.

System	Typed programs	Diverse views	Logit verifier	RCSE	Adaptive control
Direct	–	–	–	–	–
Multi-view	✓	✓	–	–	–
+ Evidence	✓	✓	–	–	–
+ Logits	✓	✓	✓	–	–
COVER-Core	✓	✓	✓	–	fixed
COVER-v2	✓	✓	✓	✓	✓
COVER-v2 + DoLa	✓	✓	✓	✓	✓

Secondary ablations should test heterogeneous verification, prompt-distribution disagreement, and relation-specific view families one at a time.

27.3 Metrics beyond official F1

Report:

- macro P/R/F1 overall and per relation;
- calls/query, generated tokens/query, and latency/query;
- F1 vs compute budget curve;
- verified-candidate yield per view;
- false-positive rejection rate of the verifier;
- null accuracy for death/stock/borders;
- numeric cluster success and relative error;
- self-consistent-error cases where repeated direct generations agree but independent views/verifier disagree;
- stopping-round distribution by relation.

28 Compliance and Safety Audit for Competition Rules

Table 9: Architecture compliance checklist.

Component	Allowed?	Notes
Frozen open-weight LLM inference	Yes	Must pass total parameter budget audit.
Multiple frozen models	Yes	Published parameter totals are summed.
Prompt/facet diversification	Yes	Inference-time prompting only.
Few-shot examples from provided train split	Yes in task precedent	Use as in-context examples/system design; never update weights.
Raw logit extraction	Yes	Inference-time model output; no training.
Contextual logit correction	Yes	Deterministic non-neural calibration; default $T = 1$.
DoLa-style intermediate-layer decoding	Architecturally compatible	Same frozen checkpoint; verify engine and budget interpretation.
Rule-based relation router	Yes	Non-neural.

Component	Allowed?	Notes
Evidence graph / deterministic score	Yes	Non-neural aggregation.
Numeric clustering and unit conversion	Yes	Non-neural processing.
Web search / Wikipedia / Wikidata lookup	No	Never allowed in prediction path.
RAG / external corpus	No	Never allowed in prediction path.
Fine-tuning / LoRA / continued training	No	Not part of architecture.
Task-trained neural verifier/probe	No	Do not train a new neural component.
MoE counted by active parameters only	No	Official rule counts total published parameters.

29 Failure Modes and Fallbacks

Table 10: Expected risks and concrete fallback behavior.

Risk	Consequence	Fallback
Raw verifier logits are overconfident	wrong candidates pass threshold	contextual correction + margin + multi-template disagreement
Same model repeats same wrong fact	false confidence from frequency	independence groups; blind verification; optional cross-family check
DoLa hurts chosen model	factual-decoding branch degrades F1	disable feature flag; architecture remains intact
Active controller over-explores awards	precision collapse	stricter verifier tier + marginal-yield stop + hard budget
Controller stops too early	recall loss	mandatory views + missingness checkpoint before final stop
Numeric responses form multiple clusters	unstable scalar prediction	more semantic views, dominant-cluster test, robust median
Null gate is over-conservative	excessive empty outputs	require independent support for empty state; tune non-neural threshold on train/internal validation
Model budget ambiguity	rule violation risk	conservative published-total count; do not enable ambiguous pairing until audited
Multimodal checkpoint complicates text-only runtime	unnecessary components/memory	use a text-only adapter or alternative checkpoint after budget audit
Partial gold for rare open-ended awards	true long-tail facts may score as false positives	precision-aware saturation; avoid unconstrained tail expansion

30 Architecture Novelty and Claim Boundaries

30.1 Ideas inherited from prior work

- ReWiSe: relation-wise object aggregation and repeated probing [3].
- Self-generated-context + divide-and-conquer LM-KBC: relation-focused context and decomposition [4].
- Atomic Self-Consistency: merge useful atomic information rather than selecting one whole answer [6].
- Chain-of-Verification: verify independently of the initial draft [7].
- SELF-DISCOVER: reasoning structure should depend on task type [8].

- Contextual calibration and MONITOR: output distributions expose prompt bias and factual instability [10, 11].
- DoLa: later-vs-earlier layer contrast can surface factual knowledge [9].
- Test-time scaling: allocate inference compute adaptively by difficulty/value [14].

30.2 What COVER-KBC v2 claims as its own system contribution

The architecture contribution is the integration around a new control abstraction, not the individual building blocks:

1. **Relation-Typed Active Set Elicitation**: each fixed relation is compiled into an executable inference program with its own discovery, verification, and stopping semantics.
2. **Independence-Aware Logit-Calibrated Atomic Verification**: evidence is tracked per candidate and per acquisition mechanism; calibrated verifier logits and prompt-distribution disagreement are fused without neural training.
3. **Coverage/Uncertainty-Guided Test-Time Control**: the system spends compute according to marginal verified yield, uncertainty, semantic gaps, redundancy, and remaining budget rather than a fixed number of generations.

Do not claim that COVER invents CoVe, DoLa, contextual calibration, self-consistency, or test-time scaling. Those are referenced mechanisms or inspirations. The paper claim is the relation-typed evidence-acquisition architecture for closed-book open-set KB completion.

31 Implementation Invariants for the Coding Phase

These rules are treated as architecture contracts during implementation:

1. The official benchmark snapshot and evaluator are immutable dependencies; code wraps them rather than editing them for convenience.
2. Every model call is logged with prompt hash, model/revision, seed/decoding config, token counts, and provenance.
3. Every candidate in the final output can be traced to one or more Evidence Graph events.
4. Verifier prompts never include the generator's free-form reasoning.
5. No external factual source is reachable from the inference path.
6. The model parameter budget is checked by a dedicated audit command before full evaluation.
7. Every experimental branch is behind configuration and has a baseline-compatible fallback.
8. Relation-specific code implements an interface; no large `if/elif` chain should leak throughout the codebase.
9. Thresholds and controller constants are stored in versioned config, not hidden in code.
10. A complete validation run can be reproduced from one command and a committed config.

32 Definition of Done Before Test Submission

System freeze checklist

The system is ready to freeze only when all conditions below hold:

- official evaluator version is current and checksummed;
- chosen neural profile passes conservative $\leq$ 32B audit;
- all six relation programs have unit and integration tests;
- validation output is evaluator-valid JSONL with no duplicate rows or missing queries;
- every ablation has a saved config and result manifest;
- final prompts and thresholds are frozen;
- no web/RAG/external KB dependency exists in runtime environment;
- final run records total calls, tokens, latency, and model revisions;

- README reproduces the final prediction command;
- paper claims match only features actually present in the frozen code.

33 One-Page Engineering Summary

What to build first

First: baseline/evaluator harness + model runtime + budget audit.
Second: relation contracts + typed programs + fixed multi-view discovery.
Third: evidence graph + A/B/C logit verifier + final selector.
Fourth: RCSE + adaptive controller/stopping.
Last: DoLa and heterogeneous-model branches only if validation demonstrates value.

One-sentence method

COVER-KBC v2 compiles each AKBC relation into a typed inference program that actively acquires structurally diverse parametric evidence, tracks atomic candidate provenance, verifies uncertain candidates with calibrated autoregressive logits, and allocates further test-time compute only while expected verified coverage gain justifies the cost and redundancy.

The architecture in one line

Contract → Typed Program → Elicit → Graph → Verify → RCSE → Act/Stop → Final Set

A Illustrative Prompts

A.1 Blind verifier

```
System:
You verify exactly one candidate against the supplied relation definition.
Do not assume the candidate is correct. Use only your internal parametric knowledge.

Subject: Germany
Relation: countryLandBordersCountry
Definition: Countries or comparable territories that share a physical land
border with the subject. Maritime-only borders are invalid.
Candidate: Sweden

A = VALID
B = INVALID
C = UNKNOWN

Return exactly one label.
```

A.2 Award missingness view

```
Subject award: <AWARD>
Verified recipients so far: <LIST>

Do not repeat the verified recipients.
Identify one coherent facet that may still contain missing recipients
(e.g. time range, recipient type, discipline/category if applicable),
then return only additional likely recipient entities from that facet.
Do not return winning works, nominees, similarly named awards, or rescinded recipients.
```

A.3 Capacity contrastive view

```
For <VENUE>, identify the highest published maximum spectator capacity.
Distinguish this from average attendance, record attendance, and any smaller
seated-only capacity. Return one integer only if you have a factual value;
otherwise return UNKNOWN.
```

B Suggested Configuration Skeleton

```yaml
experiment:
  name: cover_v2_core
  seed: 42
  max_calls_per_query: 12
  max_generated_tokens_per_query: 6000

model_profile:
  generator: <MODEL_ID>
  verifier: <MODEL_ID_OR_SAME>
  parameter_budget_audit_required: true

features:
  diverse_views: true
  evidence_graph: true
  logit_verifier: true
  contextual_calibration: true
  rcse: true
  adaptive_controller: true
  dola_view: false
  cross_model_view: false

relations:
  countryLandBordersCountry: configs/relations/borders.yaml
  personHasCityOfDeath: configs/relations/death_city.yaml
  hasCapacity: configs/relations/capacity.yaml
  awardWonBy: configs/relations/award.yaml
  companyTradesAtStockExchange: configs/relations/stock_exchange.yaml
  hasArea: configs/relations/area.yaml
```

C References

References

[1] AKBC Shared Task Organizers. *AKBC Shared Task 2026: Predicting complete knowledge base entries from language models*. Official task page, 2026. https://www.akbc.ws/2026/shared-task.html

[2] LM-KBC Organizers. *dataset2026: Dataset and baseline for the AKBC Shared Task 2026*. GitHub repository, 2026. https://github.com/lm-kbc/dataset2026

[3] E. Albert-Roulhac and A. Zouaq. ReWiSe: Relation-Wise Self-consistency for LLM Probing. In *Joint Proceedings of KBC-LM and LM-KBC @ ISWC*, CEUR-WS Vol. 4041, 2025. https://ceur-ws.org/Vol-4041/paper8.pdf

[4] J. He and S. Razniewski. Combining Self-Retrieval-Augmented Generation with Divide-and-Conquer for Language Model-based Knowledge Base Construction. In *Joint Proceedings of KBC-LM and LM-KBC @ ISWC*, CEUR-WS Vol. 4041, 2025. https://ceur-ws.org/Vol-4041/paper11.pdf

[5] X. Zhang, Q. Meng, Y. Chen, Y. Wang, and J. Bos. KnowledgeBerg: Evaluating Systematic Knowledge Coverage and Compositional Reasoning in Large Language Models. In *Findings of ACL*, 2026. https://aclanthology.org/2026.findings-acl.548/

[6] R. Thirukovalluru, Y. Huang, and B. Dhingra. Atomic Self-Consistency for Better Long Form Generations. In *Proceedings of EMNLP*, 2024. https://arxiv.org/abs/2405.13131

[7] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston. Chain-of-Verification Reduces Hallucination in Large Language Models. In *Findings of ACL*, 2024. https://aclanthology.org/2024.findings-acl.212/

[8] P. Zhou, J. Pujara, X. Ren, X. Chen, H.-T. Cheng, Q. V. Le, E. H. Chi, D. Zhou, S. Mishra, and H. S. Zheng. SELF-DISCOVER: Large Language Models Self-Compose Reasoning Structures. In *Advances in Neural Information Processing Systems*, 2024. https://openreview.net/forum?id=BROvXhmzYK

[9] Y.-S. Chuang, Y. Xie, H. Luo, Y. Kim, J. Glass, and P. He. DoLa: Decoding by Contrasting Layers Improves Factuality in Large Language Models. In *ICLR*, 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/edc36117f795ca52a0cbf6a7b3882859-Abstract-Conference.html

[10] T. Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh. Calibrate Before Use: Improving Few-Shot Performance of Language Models. In *ICML*, 2021. https://arxiv.org/abs/2102.09690

[11] W. Wang, B. Haddow, A. Birch, and W. Peng. Assessing Factual Reliability of Large Language Model Knowledge. In *NAACL*, 2024. https://aclanthology.org/2024.naacl-long.46/

[12] L. Kuhn, Y. Gal, and S. Farquhar. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. ICLR, 2023. https://arxiv.org/abs/2302.09664

[13] H. Tan, F. Sun, S. Liu, D. Su, Q. Cao, X. Chen, J. Wang, X. Cai, Y. Wang, H. Shen, and X. Cheng. Too Consistent to Detect: A Study of Self-Consistent Errors in LLMs. In *Proceedings of EMNLP*, 2025. https://aclanthology.org/2025.emnlp-main.238/

[14] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM Test-Time Compute Optimally Can be More Effective than Scaling Parameters for Reasoning. ICLR, 2025. https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b623663fd9b874366f3ce019fdfdd44-Abstract-Conference.html

[15] Mistral AI. *Mistral-Small-3.2-24B-Instruct-2506 model card*. https://huggingface.co/mistralai/Mistral-Small-3.2-24B-Instruct-2506

[16] Qwen Team. *Qwen3.5-27B model card*. https://huggingface.co/Qwen/Qwen3.5-27B

[17] Qwen Team. *Qwen3.5-4B model card*. https://huggingface.co/Qwen/Qwen3.5-4B